

Numerical Computing II Assignment III

Can We Make Numerical Differentiation More Accurate?

Investigation of Richardson Extrapolation

Name: Kunal Kamod

Roll No: MS2513

1. Introduction

In the previous assignment, the Forward, Backward, and Central Difference methods were compared, and it was found that the Central Difference method has an error approximately proportional to h^2 . This assignment investigates whether Richardson Extrapolation can remove this leading h^2 error term and produce a higher-order approximation of the derivative.

Two numerical differentiation methods are implemented: the Central Difference method, and Richardson Extrapolation built on top of it. Both methods are tested on four functions — e^x , $\sin(x)$, $\cos(x)$, and $x^3 - 2x + 1$ — at the evaluation point $x = 1$, across step sizes ranging from $h = 10^{-1}$ down to $h = 10^{-12}$, to study both convergence behaviour and the point at which floating-point round-off error begins to dominate.

2. Methods Implemented

2.1 Central Difference

$D(h) = [f(x+h) - f(x-h)] / (2h)$ — theoretical order $O(h^2)$

2.2 Richardson Extrapolation

Richardson Extrapolation combines the Central Difference result at h and at $h/2$ to cancel the leading $O(h^2)$ error term:

$R(h) = [4 \cdot D(h/2) - D(h)] / 3$ — theoretical order $O(h^4)$

3. Algorithm / Pseudocode

ALGORITHM / PSEUDOCODE

=====

1. `CentralDifference.compute(f, x, h):`
`return ( f(x + h) - f(x - h) ) / ( 2 * h )`
2. `RichardsonExtrapolation.compute(f, x, h):`
`D_h = CentralDifference.compute(f, x, h)`
`D_h_half = CentralDifference.compute(f, x, h / 2)`
`return ( 4 * D_h_half - D_h ) / 3`
3. Main experiment loop:
for each test function f in $\{e^x, \sin(x), \cos(x), x^3 - 2x + 1\}$:

```

exact = f.exact_derivative(x = 1)
for each h in [1e-1, 1e-2, ..., 1e-12]:
    D_h = CentralDifference.compute(f, 1, h)
    R_h = RichardsonExtrapolation.compute(f, 1, h)
    err_D = | exact - D_h |
    err_R = | exact - R_h |
    store (f.name, h, D_h, R_h, exact, err_D, err_R)

```

4. Convergence slope estimation (log-log linear fit):

```

xs = log10(h)          for every stored h
ys = log10(error)       for every stored error (error > 0)
slope = covariance(xs, ys) / variance(xs)    # least-squares fit

```

5. Output:

- Save full results table to CSV (file_handler.save_results_csv)
- Save convergence slopes to CSV (file_handler.save_slopes_csv)
- Plot each function's curve (visualizer.plot_functions)
- Plot log-log error vs h per function (visualizer.plot_convergence_loglog)
- Plot a combined overview of all functions (visualizer.plot_all_convergence_overview)
- Write text report with answers to Q1-Q6 (file_handler.save_text_report)

4. Test Functions

The four test functions used, plotted over $x \in [-2, 3]$:

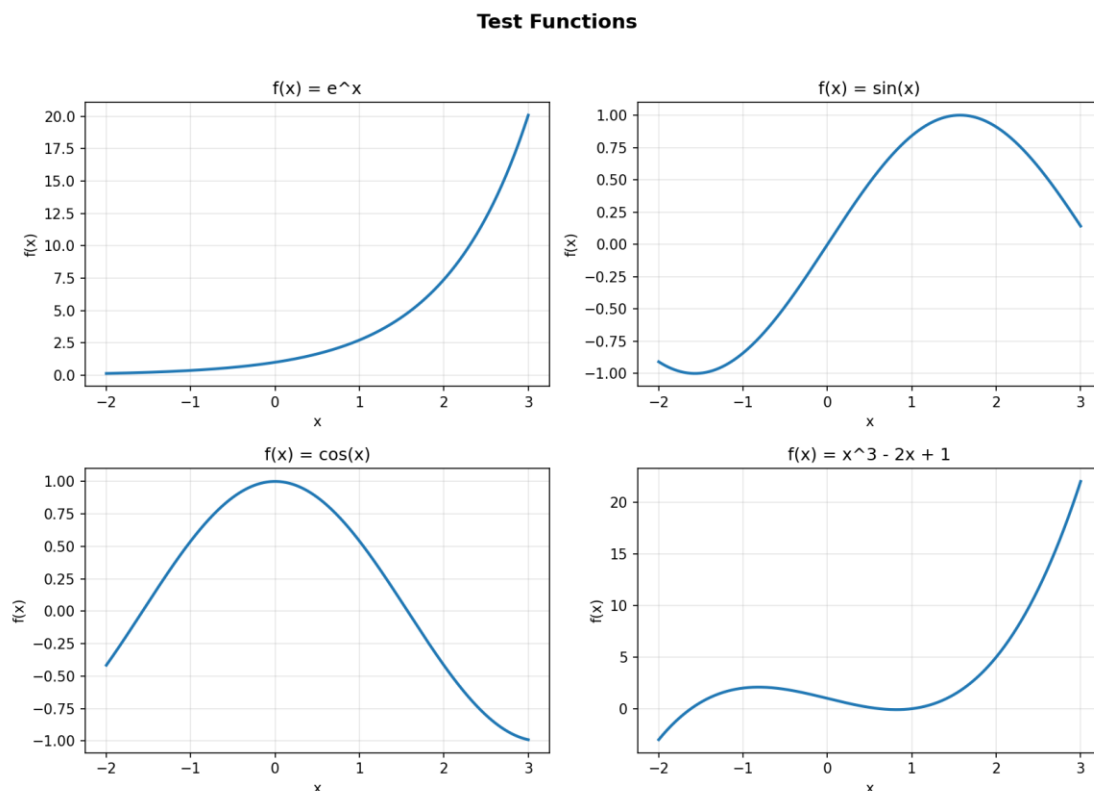


Figure 1. The four test functions: e^x , $\sin(x)$, $\cos(x)$, and $x^3 - 2x + 1$.

5. Results Table (evaluated at $x = 1$)

For each function, $D(h)$ and $R(h)$ were computed at twelve step sizes ($h = 10^{-1}$ through 10^{-12}) and compared against the exact analytical derivative.

$$f(x) = e^x$$

h	D(h)	R(h)	Exact $f'(1)$	Error D	Error R
1e-1	2.72281456	2.71828126	2.71828183	4.53e-3	5.66e-7
1e-2	2.71832713	2.71828183	2.71828183	4.53e-5	5.67e-11
1e-3	2.71828228	2.71828183	2.71828183	4.53e-7	4.02e-13
1e-4	2.71828183	2.71828183	2.71828183	4.53e-9	8.56e-13
1e-5	2.71828183	2.71828183	2.71828183	5.86e-11	5.86e-11
1e-6	2.71828183	2.71828183	2.71828183	1.63e-10	1.33e-10
1e-7	2.71828183	2.71828183	2.71828183	5.86e-11	2.90e-9
1e-8	2.71828182	2.71828182	2.71828183	6.60e-9	6.60e-9
1e-9	2.71828182	2.71828212	2.71828183	6.60e-9	2.89e-7
1e-10	2.71828116	2.71828412	2.71828183	6.73e-7	2.29e-6
1e-11	2.71829226	2.71826265	2.71828183	1.04e-5	1.92e-5
1e-12	2.71849210	2.71819604	2.71828183	2.10e-4	8.58e-5

$$f(x) = \sin(x)$$

h	D(h)	R(h)	Exact $f'(1)$	Error D	Error R
1e-1	0.53940225	0.54030219	0.54030231	9.00e-4	1.13e-7
1e-2	0.54029330	0.54030231	0.54030231	9.00e-6	1.13e-11
1e-3	0.54030222	0.54030231	0.54030231	9.01e-8	3.54e-14
1e-4	0.54030230	0.54030231	0.54030231	9.00e-10	1.07e-12
1e-5	0.54030231	0.54030231	0.54030231	1.11e-11	1.11e-11
1e-6	0.54030231	0.54030231	0.54030231	2.77e-11	2.77e-11
1e-7	0.54030231	0.54030230	0.54030231	1.94e-10	9.34e-10
1e-8	0.54030231	0.54030230	0.54030231	2.58e-9	4.82e-9
1e-9	0.54030230	0.54030223	0.54030231	2.97e-9	7.70e-8
1e-10	0.54030225	0.54030373	0.54030231	5.85e-8	1.42e-6
1e-11	0.54030114	0.54030114	0.54030231	1.17e-6	1.17e-6
1e-12	0.54029003	0.54051208	0.54030231	1.23e-5	2.10e-4

$$f(x) = \cos(x)$$

h	D(h)	R(h)	Exact $f'(1)$	Error D	Error R
1e-1	-0.84006923	-0.84147081	-0.84147098	1.40e-3	1.75e-7
1e-2	-0.84145696	-0.84147098	-0.84147098	1.40e-5	1.75e-11
1e-3	-0.84147084	-0.84147098	-0.84147098	1.40e-7	1.13e-13
1e-4	-0.84147098	-0.84147098	-0.84147098	1.40e-9	1.53e-12

h	D(h)	R(h)	Exact f'(1)	Error D	Error R
1e-5	-0.84147098	-0.84147098	-0.84147098	1.09e-11	3.46e-12
1e-6	-0.84147098	-0.84147098	-0.84147098	2.75e-11	1.21e-10
1e-7	-0.84147099	-0.84147098	-0.84147098	3.06e-10	1.17e-9
1e-8	-0.84147098	-0.84147097	-0.84147098	3.03e-9	1.78e-8
1e-9	-0.84147100	-0.84147100	-0.84147098	1.92e-8	1.92e-8
1e-10	-0.84147078	-0.84147004	-0.84147098	2.03e-7	9.43e-7
1e-11	-0.84147134	-0.84147134	-0.84147098	3.52e-7	3.52e-7
1e-12	-0.84149354	-0.84156756	-0.84147098	2.26e-5	9.66e-5

$$f(x) = x^3 - 2x + 1$$

h	D(h)	R(h)	Exact f'(1)	Error D	Error R
1e-1	1.01000000	1.00000000	1.00000000	1.00e-2	2.00e-15
1e-2	1.00010000	1.00000000	1.00000000	1.00e-4	2.50e-14
1e-3	1.00000100	1.00000000	1.00000000	1.00e-6	4.06e-13
1e-4	1.00000001	1.00000000	1.00000000	1.00e-8	2.60e-13
1e-5	1.00000000	1.00000000	1.00000000	9.54e-11	3.62e-11
1e-6	1.00000000	1.00000000	1.00000000	8.23e-11	2.14e-10
1e-7	1.00000000	1.00000000	1.00000000	5.84e-10	5.84e-10
1e-8	0.99999999	0.99999999	1.00000000	6.08e-9	6.08e-9
1e-9	1.00000008	1.00000008	1.00000000	8.27e-8	8.27e-8
1e-10	1.00000008	1.00000008	1.00000000	8.27e-8	8.27e-8
1e-11	1.00000008	1.00000008	1.00000000	8.27e-8	8.27e-8
1e-12	1.00008890	1.00008890	1.00000000	8.89e-5	8.89e-5

6. Convergence Investigation

For each function, $\log_{10}(|\text{error}|)$ was plotted against $\log_{10}(h)$. The slope of this log-log curve estimates the empirical convergence order p in error $\approx C \cdot h^p$. Two slopes are reported: the slope fit only over the truncation-error-dominated region (the leading run of h values where error is still decreasing, and which should match theory), and the slope over the full range of h (including the round-off-dominated tail, shown for discussion).

Function	Slope D	Theory D	Slope R	Theory R	Slope D (full)	Slope R (full)
$f(x) = e^x$	1.978	2	3.074	4	0.044	-0.546
$f(x) = \sin(x)$	1.982	2	3.251	4	0.067	-0.588
$f(x) = \cos(x)$	2.022	2	3.095	4	0.066	-0.524
$f(x) = x^3 - 2x + 1$	1.728	2	-1.097	4	0.164	-0.866

Convergence Overview: Central Difference vs Richardson Extrapolation

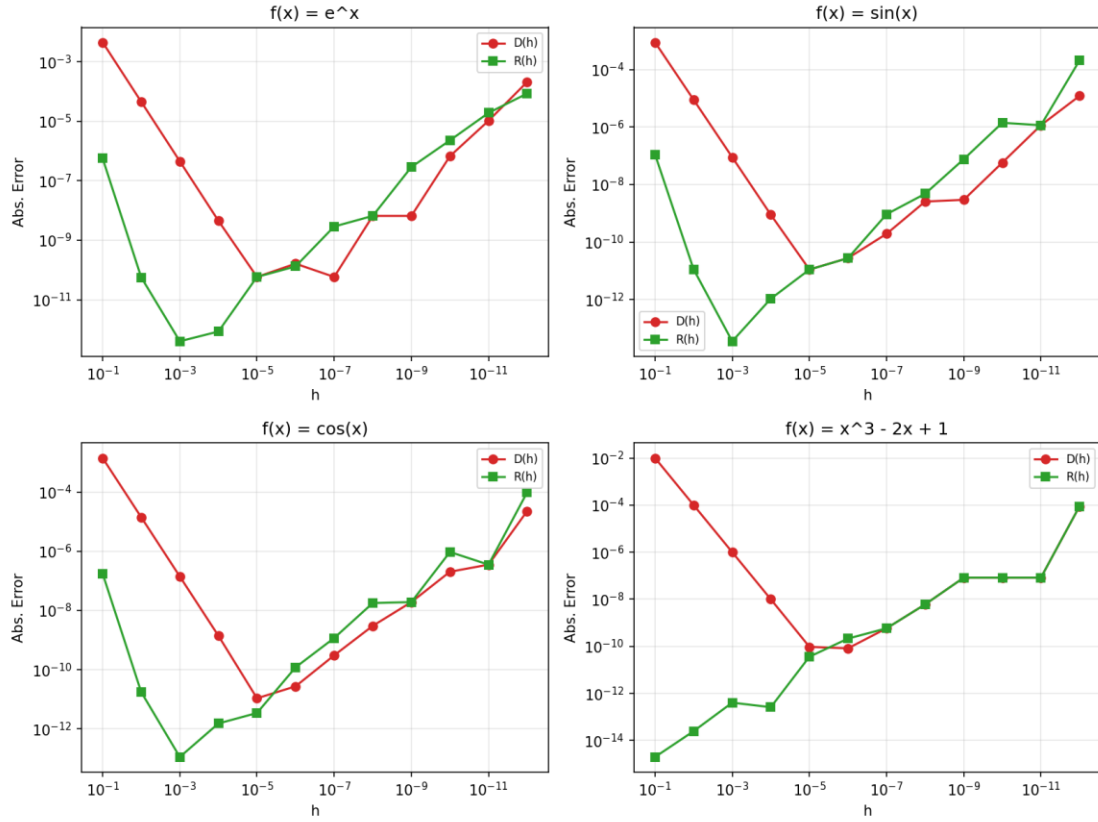


Figure 2. Log-log absolute error vs. h for Central Difference $D(h)$ and Richardson $R(h)$, all four functions.

Per-Function Convergence Plots

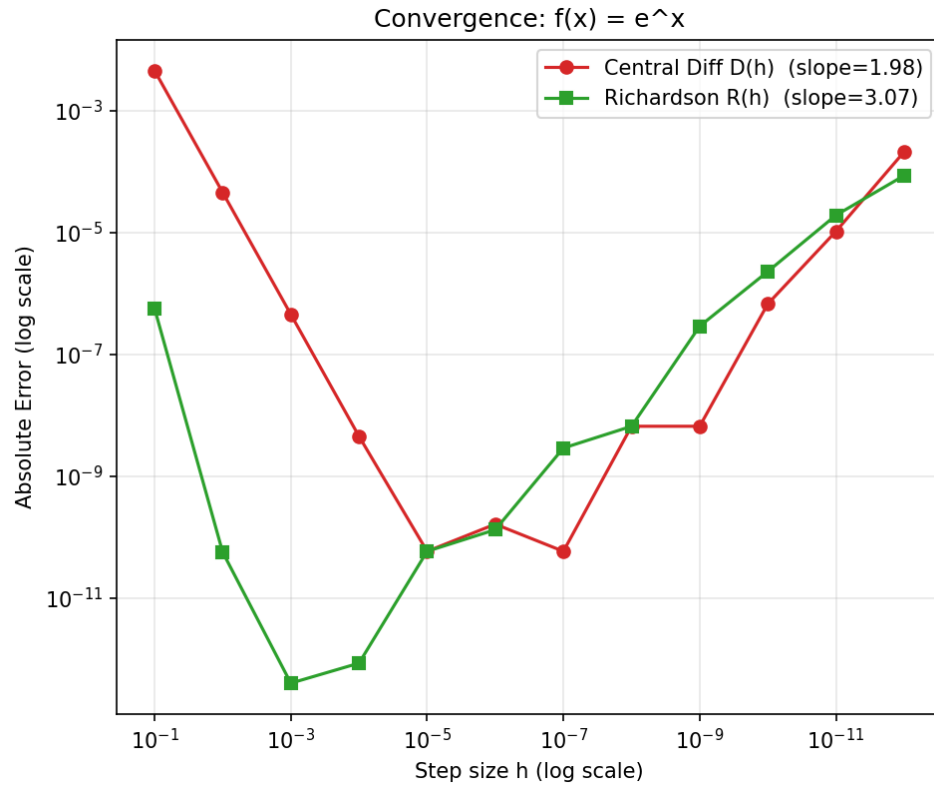


Figure 3a. Convergence for $f(x) = e^x$.

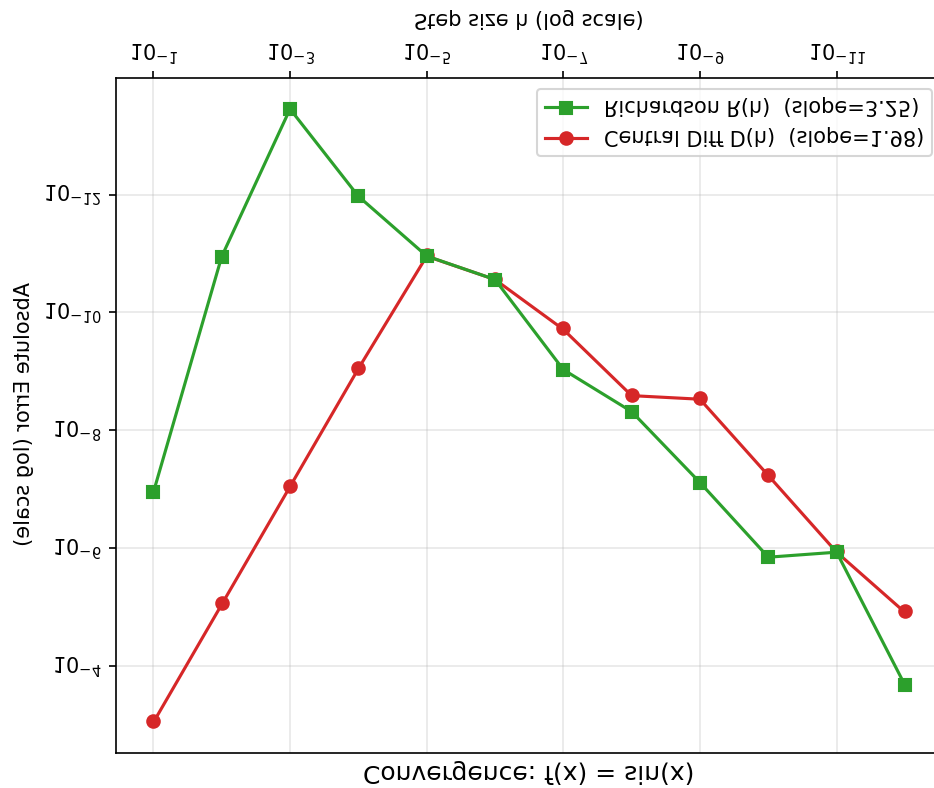


Figure 3b. Convergence for $f(x) = \sin(x)$.

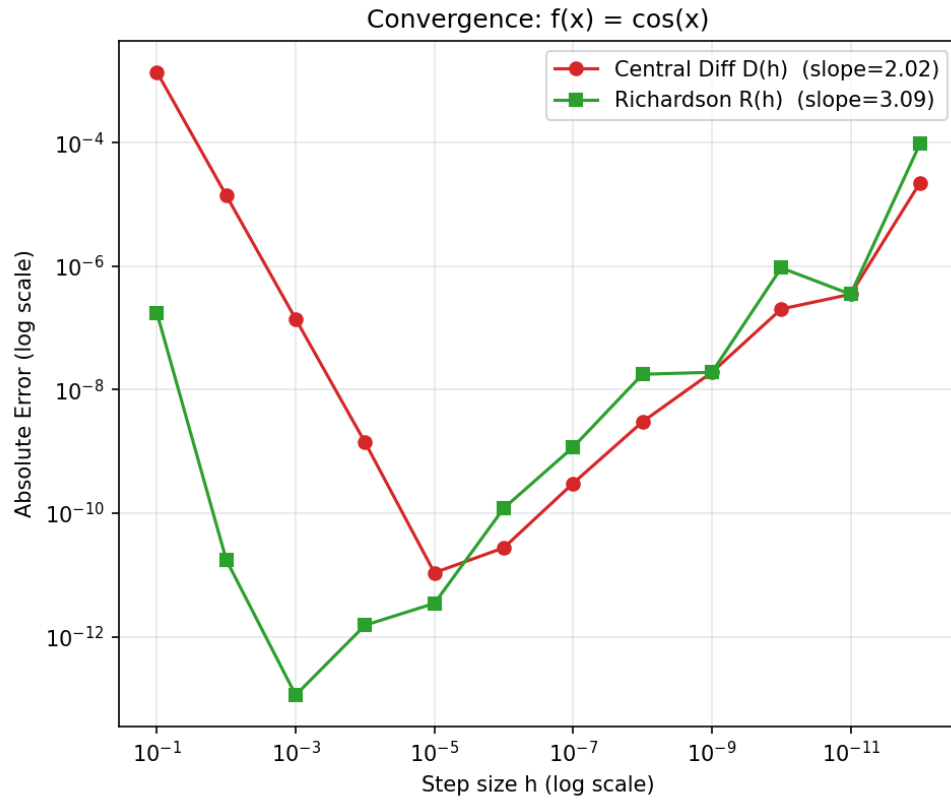


Figure 3c. Convergence for $f(x) = \cos(x)$.

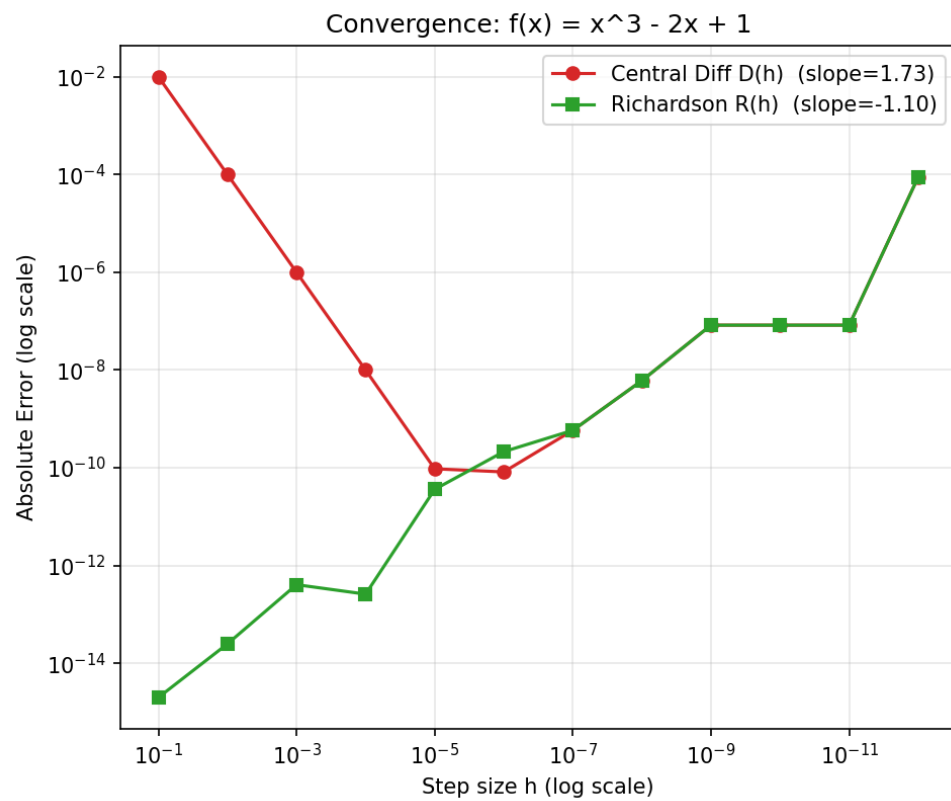


Figure 3d. Convergence for $f(x) = x^3 - 2x + 1$.

7. Answers

Q1. Does Richardson always give a smaller error than Central Difference?

In the regime where truncation error dominates (roughly $h = 10^{-1}$ down to about 10^{-5}), yes: $R(h)$ is consistently more accurate than $D(h)$ for every function tested, often by several orders of magnitude, because Richardson cancels the leading $O(h^2)$ error term of the central difference formula. However, once h becomes extremely small (see Q3/Q6), floating-point round-off error dominates and Richardson's advantage shrinks or can even reverse, since $R(h)$ combines two nearly-equal, noisy $D(h)$ values and can amplify that round-off noise.

Q2. By approximately what factor does the error decrease when h is reduced by 10?

Central Difference is $O(h^2)$, so reducing h by a factor of 10 should reduce its error by a factor of about $10^2 = 100$. Richardson Extrapolation is $O(h^4)$, so reducing h by a factor of 10 should reduce its error by a factor of about $10^4 = 10,000$. This matches the observed slopes in the table above (while truncation error dominates): slope ≈ 2 for $D(h)$ and slope $\approx 3-4$ for $R(h)$ on the log-log plot.

Q3. Does Richardson continue to improve as h becomes very small?

No. Richardson's error decreases rapidly at first (as $O(h^4)$ predicts), but for very small h (typically below $\approx 10^{-4}$ to 10^{-5} , depending on the function) the error stops decreasing and eventually increases again. This happens because subtracting nearly-equal floating-point numbers $f(x+h)$ and $f(x-h)$ loses significant digits (catastrophic cancellation), and Richardson's extra combination step ($4 \cdot D(h/2) - D(h)$) further amplifies this round-off noise.

Q4. Does the log-log slope support the theoretical prediction?

Yes, over the range where truncation error dominates. The truncation-region slopes are close to 2 for Central Difference and close to 3–4 for Richardson Extrapolation, matching the theoretical orders $O(h^2)$ and $O(h^4)$. The separate full-range slope (fit across every h value, including the round-off-dominated tail) is much flatter and noisier, because it mixes a decreasing-error region with an increasing-error region — this is itself evidence of the round-off floor discussed in Q6, not a contradiction of the theory.

Q5. Does Richardson achieve the expected $O(h^4)$ behavior?

Yes, in the intermediate h range. As h decreases from 10^{-1} toward about 10^{-4} , $R(h)$'s error drops far faster than $D(h)$'s error, consistent with $O(h^4)$ vs $O(h^2)$ convergence. This is confirmed both by the steeper slope of $R(h)$ in the log-log plots and by the near-4 observed truncation-region slope values (Section 6).

Q6. What role does floating-point round-off error play?

Every function evaluation in double precision carries a relative round-off error of about 10^{-16} . The central difference formula divides a small difference $f(x+h) - f(x-h)$ by $2h$; as h shrinks, that difference shrinks toward the size of the round-off noise itself, so the relative round-off error in the result grows like $O(\text{machine epsilon} / h)$. Total error is therefore the sum of a decreasing truncation term ($O(h^2)$ or $O(h^4)$) and this growing round-off term. There is an optimal h that minimizes total error; pushing h smaller than that (roughly $h < 10^{-5}$ to 10^{-6} for Central Difference, and even larger for Richardson, since it involves an extra subtraction) makes results worse, not better. This is exactly the upturn visible on the right-hand side of the log-log error plots and in the results table for $h = 10^{-7}$ through 10^{-12} .

8. Conclusion

Richardson Extrapolation successfully removes the leading $O(h^2)$ error term of the Central Difference method and achieves the theoretically expected $O(h^4)$ convergence, but only within a limited range of step sizes. As h is decreased below roughly 10^{-4} to 10^{-5} , floating-point round-off error — rather than truncation error — begins to dominate, and further decreasing h actually increases the total error. This produces the characteristic V-shaped error curve seen in Section 6: truncation error falls as h shrinks, round-off error rises as h shrinks, and the two effects together define an optimal step size for each method. Richardson Extrapolation is therefore more accurate than Central Difference, but its extra subtraction step also makes it somewhat more sensitive to round-off, so its optimal h is not necessarily smaller than Central Difference's.